

## Intermediate Programming Concepts

# Linked List

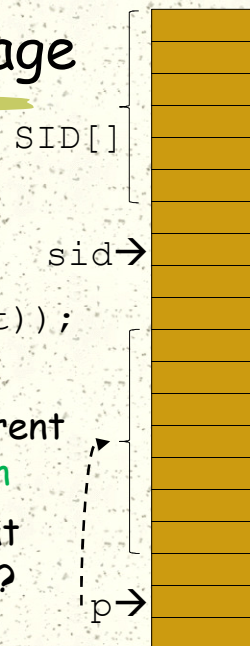
Dr. Gang Qu

University of Maryland, College Park  
gangqu@umd.edu

## Data and Memory Storage

```
int sid;  
int SID[6];  
int *p;  
p=(int *)malloc(7*sizeof(int));
```

- # Q1. Can (array) data be of different data types? Answer: **struct, union**
- # Q2. Can (array) data be stored at non-consecutive physical address? Answer: **pointer, linked list**



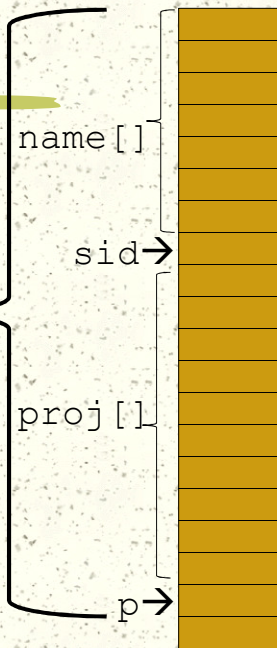
MeshSec Lab

© Gang Qu (gangqu@umd.edu)

2

## Q1 - struct

```
struct student
{ char name[40];
  int sid;
  float proj[3];
  int *p;
};
struct student Joe;
struct student *S;
S=(struct student*) malloc
(sizeof(struct student));
```



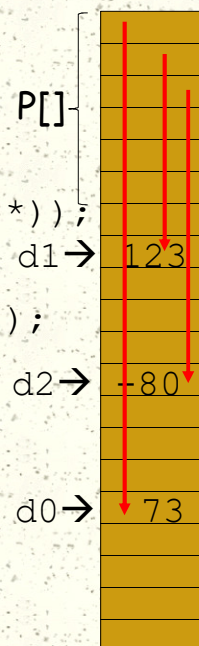
MeshSec Lab

@ Gang Qu (gangqu@umd.edu)

3

## Q2. - Pointer Array

```
int **p;
p=(int **)malloc(6*sizeof(int *));
for (int i=0; i<6; i++)
p[i]=(int *)malloc(sizeof(int));
*p[0] = 73;
*p[1] = 123;
*p[2] = -80;
```



# Q2. Can (array) data be stored at non-consecutive physical address?  
Answer: **pointer, linked list**



MeshSec Lab

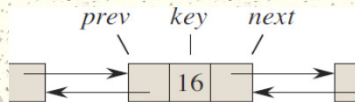
@ Gang Qu (gangqu@umd.edu)

4

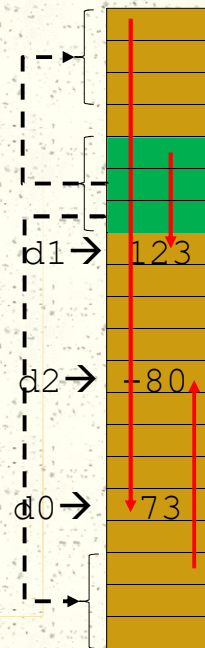


## Putting These Together

```
struct node {
    int *key;
    struct node *next, *prev;
}
```



```
typedef struct node NODE;
NODE *tmp;
tmp = (NODE *)
    malloc(sizeof(NODE));
if (tmp == NULL) // fails
```

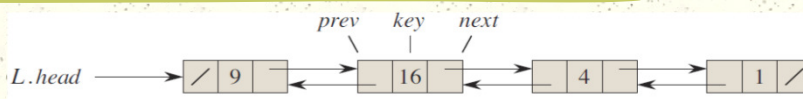


MeshSec Lab

@ Gang Qu (gangqu@umd.edu)

5

## Linked List: Node



# Define the node in the list

```
typedef struct node {
    int *key;
    struct node *next, *prev;
} NODE;
```

1. linkedlist.c  
2. linkedlist2.c

# Declare a new NODE

```
NODE *tmp;
tmp = (NODE *)malloc(sizeof(NODE));
if (tmp == NULL) //fail, report error, exit
```



MeshSec Lab

@ Gang Qu (gangqu@umd.edu)

6

## Linked List: Root

- \* Create the ROOT node in the list

```
typedef struct root {
    long num;
    NODE *head, *tail;
}ROOT;
```

1. linkedlist.c
2. linkedlist2.c

- \* initialize an empty linked list, or create a root:

```
ROOT * make_root(void) // why function here?
{
    ROOT *tmp;
    tmp = (ROOT *) malloc(sizeof(ROOT));
    if (tmp == NULL) return tmp;
    tmp->head=NULL; tmp->tail=NULL; tmp->num=0;
    return tmp;
}
```

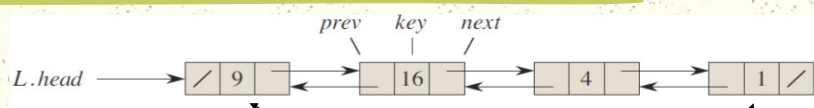


MeshSec Lab

@ Gang Qu (gangqu@umd.edu)

7

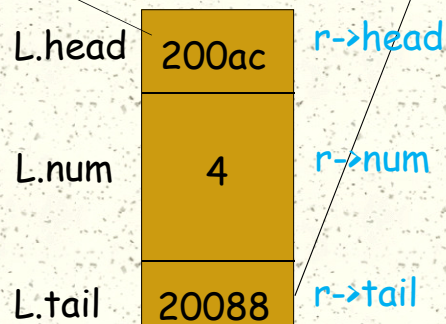
## A Linked List with Root



ROOT L;

ROOT \*r = &L;

. dot access vs  
-> pointer access



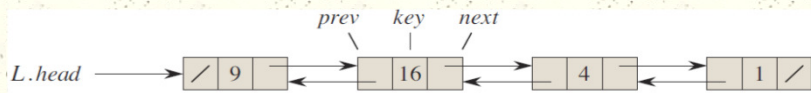
MeshSec Lab

@ Gang Qu (gangqu@umd.edu)

8



## Linked List Operation: Traverse



```

NODE x;  if (L.head != NULL)
for (x=*L.head; &x!=NULL; x=*x.next)
    x.key = x.key * 2;

int i;
for (i=0; i<n; i++)
    A[i] = A[i] * 2;

NODE *t;
for (t=L.head; t!=NULL; t=t->next)
    t->key = t->key * 2;
    
```

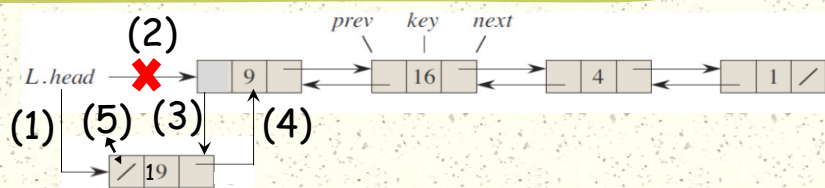


MeshSec Lab

© Gang Qu (gangqu@umd.edu)

9

## Linked List Operation: Insert



### # Insert at head

```

node *t;
t->next = L.head;      //(4)
t->prev = NULL;        //(5)
L.head->prev = t;      //(3)
L.head = t;            //(1) & (2)
L.num++;
    
```

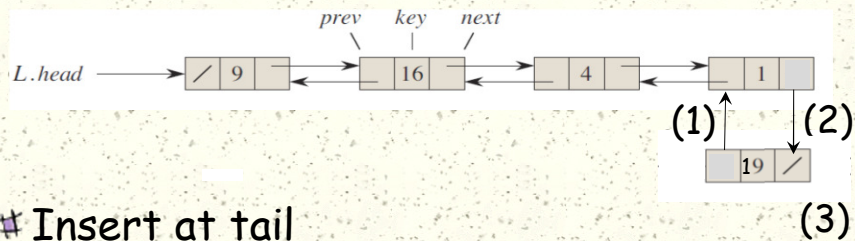


MeshSec Lab

© Gang Qu (gangqu@umd.edu)

10

## Linked List Operation: Insert



### # Insert at tail

```
node *t;
L.tail->next = t;    //(2)
t->prev = L.tail;    //(1)
t->next = NULL;      //(3)
L.num++; L.tail = t; // update root L
```

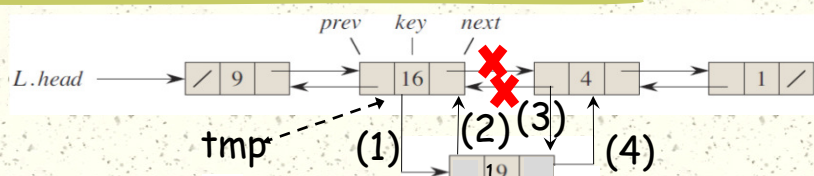


MeshSec Lab

© Gang Qu (gangqu@umd.edu)

11

## Linked List Operation: Insert



### # Insert in the middle (after tmp)

```
node *t;
tmp->next->prev = t; //(3) ✗
t->next = tmp->next; //(4)
tmp->next = t;       //(1) ✗
t->prev = tmp;       //(2)
```



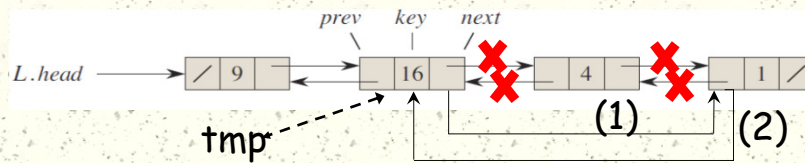
MeshSec Lab

© Gang Qu (gangqu@umd.edu)

12



## Linked List Operation: Delete



### # Delete in the middle (after tmp)

```
tmp->next = tmp->next->next; // (1)
```

```
tmp->next -> prev = tmp; // (2)
```

```
L.num--;
```

```
// special cases: L.num = 0 or 1
```

```
// special cases: delete the head/tail
```



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

13

## Linked List: More Concepts

### # Singly linked list vs. doubly linked list

- ▀ traverse the linked list in one vs. both directions!

### # Beyond singly and doubly linked list

- ▀ More neighbors like left, right, up, down in a grid.

### # The "runner" technique

- ▀ Use two (or more) pointers to traverse the linked list at the same time, with one ahead of the other.
- ▀ Example: interleaving  $a_1 \rightarrow a_2 \rightarrow a_3 \rightarrow \dots \rightarrow a_n \rightarrow b_1 \rightarrow b_2 \rightarrow \dots \rightarrow b_n$  to  $a_1 \rightarrow b_1 \rightarrow a_2 \rightarrow b_2 \rightarrow \dots \rightarrow a_n \rightarrow b_n$

### # Pointer enables many recursive approaches.



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

14